function_gen
从 RTL 到版图的完整练习

题目、要求与流程说明

题目	y = a・b・cos(2πc/4096) / ((a+d)・4096)，200 MHz，误差 ±4 LSB
流程	仿真、综合、形式验证、布局布线、版图验证、时序签核、后仿
工具	VCS、Verdi、Design Compiler、Formality、IC Compiler II、Calibre、PrimeTime
工艺	0.18 µm 标准单元库（自备，不随仓库提供）
分支	main：题目版；solution：跑通的完整版

目录

1. 这个项目是什么

- 1.1 两个分支
- 1.2 仓库里没有什么

2. 题目

- 2.1 要实现的函数
- 2.2 要求
- 2.3 需要你自己定的事

3. 需要准备的环境

- 3.1 工具
- 3.2 工艺库（自备）
- 3.3 开工检查

4. 仓库结构和约定

- 4.1 目录
- 4.2 需要你动的文件
- 4.3 testbench 的约定

5. 流程：十个步骤

- 5.1 第 1 步：RTL 仿真
- 5.2 第 2 步：逻辑综合
- 5.3 第 3 步：综合后形式验证
- 5.4 第 4 步：综合后仿真
- 5.5 第 5 步：布局布线
- 5.6 第 6 步：布局布线后的形式验证
- 5.7 第 7 步：版图验证
- 5.8 第 8、9 步：寄生参数和时序签核
- 5.9 第 10 步：后仿

6. 题目版留给你的问题

- 6.1 前端
- 6.2 后端
- 6.3 做题的顺序建议

7. 参考结果

8. 说明

1. 这个项目是什么

用一道完整的小题目，把数字集成电路从 RTL 到版图的全流程走一遍：自己设计一个定点运算电路，写 testbench 验证，综合，形式验证，布局布线，版图验证，时序签核，最后带着真实延迟再仿真一次。

题目不大（最终约 6000 个标准单元），但 200 MHz 的时序要求在 0.18 µm 工艺上并不宽松，前端的流水线划分和后端的每一步都要认真对待。

1.1 两个分支

分支	内容	适合谁
`main`（题目版）	题目说明、目录结构、能运行的基础流程脚本、一份验收用 testbench。**没有 RTL，没有时序约束**，后端脚本只做最基本的事	想自己做一遍的人
`solution`（完整版）	一份跑通全流程的参考设计：RTL、约束、补全后的脚本、结果数据	做完之后对照，或只想看流程怎么搭的人

建议先在 `main` 上自己做，卡住时看本文第 6 章的提示，实在过不去再翻 `solution`。

1.2 仓库里没有什么

标准单元库和 PDK 受工艺厂保密协议约束，**不随仓库提供**；由它们生成的版图、门级网表、时序报告、延迟文件也不在仓库里。EDA 工具同样需要自备。第 3 章列出了需要准备的东西。

2. 题目

2.1 要实现的函数

$$y = \frac{a \cdot b \cdot \cos(2\pi \cdot c / 4096)}{(a + d) \cdot 4096}$$

2.2 要求

项目	要求
数据格式	a、b、c、d 为 12 位无符号整数；y 为 13 位有符号数，表示小数（真实值 = y / 4096）
端口	输入**有且仅有** `a[11:0]`、`b[11:0]`、`c[11:0]`、`e`、`clk`；输出 `y[12:0]`
精度	误差不超过 $\pm 2^{-10}$，即 y 的 ±4 个最低位
时钟	200 MHz（周期 5 ns）
时钟不确定度	建立时间 1 ns，保持时间 300 ps
输入方式	a、b、c 在时钟上升沿输入，每个周期一组；d 通过同步输入端口 `e` 写入
优化目标	满足以上条件的前提下，让输入到输出的延迟拍数（latency）尽量小

2.3 需要你自己定的事

题目有意留了几处没有规定，做设计的人要自己定下来并写进报告：

- **d 怎么从一根线 `e` 送进来。** 没有 `d[11:0]` 端口，也没有复位、使能端口。你要设计一个串行格式：怎么表示一帧的开始，位序是什么，接收完成后新的 d 从哪一拍开始生效，上电后没有复位信号时电路怎么进入确定状态。
- **a + d = 0 时输出什么。** 数学上没有定义，电路必须有确定的行为。
- **舍入方式。** 截断还是四舍五入，在哪一级做。这直接影响误差预算。
- **latency 是多少。** 它是优化目标，也是 testbench 对齐期望值要用的数。

> **关于 y 的范围**
>
> a/(a+d) 不超过 1，b/4096 小于 1，余弦在 ±1 之间，所以 y 的真实值严格在 (−1, 1) 内，13 位有符号数放得下，正常情况下不会溢出。

3. 需要准备的环境

3.1 工具

仓库里的脚本是按 Synopsys 的前后端工具加 Calibre 写的，在下列版本上跑通过。别的版本一般也能用，个别选项名可能要调整。

用途	工具	跑通时的版本
仿真、看波形	VCS、Verdi	X-2025.06
逻辑综合	Design Compiler	X-2025.06
形式验证	Formality	X-2025.06
建物理库	IC Compiler II 库管理器	V-2023.12
布局布线	IC Compiler II	X-2025.06
版图验证	Calibre（DRC、天线、LVS）	2025
静态时序分析	PrimeTime	X-2025.06

系统是 RHEL 8。脚本内部用 bash，登录 shell 是 tcsh 也不影响 `./run.sh` 和 `make`。

3.2 工艺库（自备）

需要一套 0.18 µm 的标准单元库及配套 PDK。脚本默认的配置叫 `tsmc180`，对应一套 1.8 V、7 轨、6 层金属的库；用别的库时，在 `common/env.sh` 里照着这一段改路径和单元名。

文件	谁用	说明
三个工艺角的时序库 `.db`	DC、Formality、ICC2、PrimeTime	最慢、典型、最快
标准单元 LEF	库管理器	单元外框和引脚
工艺文件 `.tf`	库管理器、ICC2	层定义和布线规则
连线寄生模型 TLU+ 及层映射	ICC2	估算和提取连线电阻电容
标准单元 Verilog 模型	VCS	门级仿真
标准单元 GDS	ICC2 导出版图时合并	版图验证需要完整版图
标准单元 SPICE 网表	LVS	网表一侧的晶体管级描述
Calibre 的 DRC、天线、LVS 规则文件	Calibre	

另有三个含设计规则数值的小文件不进仓库，仓库里只有模板，需要对照自己的 PDK 填写：

```
4_icc2/icc2_files/gdsout_text.map.example     导出 GDS 时文字标签的层映射
4_icc2/icc2_files/text_layers.tcl.example     各层金属的引脚文字层号
4_icc2/icc2_files/antenna.tcl.example         天线规则（只在 solution 分支）
```

3.3 开工检查

改好 `flow.cfg` 里的库路径后运行：

```
./check_env.sh
```

它会列出工具、库文件、设计文件哪些齐了、哪些缺。

4. 仓库结构和约定

4.1 目录

目录按流程顺序编号，每个目录是一个步骤，各自有一个 `run.sh` 或 `makefile`。

```
flow.cfg                    全流程唯一需要改的配置：库路径，顶层名，时钟周期，利用率
check_env.sh                开工检查
common/env.sh               由 flow.cfg 推导出各工具要用的文件路径和单元名
common/sim.mk               三次仿真共用的 VCS 规则
0_lib/                      把 LEF + 工艺文件 + 时序库建成 ICC2 用的参考库
0_simulation_pre/           RTL 仿真；  rtl/ 放设计，tb/ 放 testbench
1_dc/                       逻辑综合；  tcl/top.sdc 是全流程共用的时序约束
2_formality_postdc/         形式验证：RTL 对综合网表
3_simulation_postdc/        综合网表 + SDF 仿真
4_icc2/                     布局布线；  scripts/ 下按阶段分成 00 ~ 08 九个脚本
5_DRC/  5_LVS/              Calibre 版图验证
6_formality_postlayout/     形式验证：综合网表对布局布线后的网表
7_StarRC/                   寄生参数（没有 StarRC 工艺文件时汇集 ICC2 导出的 SPEF）
8_pt/                       PrimeTime 三个工艺角的静态时序分析，写出后仿用的 SDF
9_simulation_postlayout/    布局布线后的网表 + SDF 仿真
```

4.2 需要你动的文件

文件	做什么
`flow.cfg`	库的根目录、时钟周期、利用率
`common/env.sh`	`RTL_FILES` 列出你的全部 `.v`；库文件名、单元名和你的库不一致时在这里改
`0_simulation_pre/rtl/`	你的设计。顶层模块名 `function_gen`
`0_simulation_pre/filelist.f`	仿真要编译的文件
`0_simulation_pre/tb/`	testbench
`1_dc/tcl/top.sdc`	时序约束（题目版里是待填的模板）
`4_icc2/scripts/*.tcl`	布局布线各阶段，题目版里是基础版本

4.3 testbench 的约定

后面两次门级仿真会复用同一份 testbench，所以它要遵守几条约定：

- 顶层模块名 `tb_function_gen`，被测电路的实例名 `dut`。
- 编译开关：`FSDB`（写波形）、`CLK_HALF`（半周期，单位 ns）、`SDF_FILE`（门级仿真时要反标的延迟文件）。
- 运行参数 `+n=`：随机向量的组数。
- 结尾打印一行统计，以及 `TEST PASSED` 或 `TEST FAILED`，脚本靠这一行判断结果。

题目版自带的 testbench 已经按这些约定写好，它用实数算出期望值，延迟若干拍后与输出比较。其中有两处和具体设计有关，需要换成你自己的：延迟拍数 `LATENCY`，以及写入 d 的那个任务所假设的串行格式。

> **写 testbench 的三个容易出错的地方**
>
> 1. 输入在时钟下降沿改变，不要贴着上升沿，否则门级仿真会报建立保持时间违例。
> 2. 用实数算期望值时，先把每个整数输入赋给实数变量再运算，不要在一个表达式里混用整数和实数。
> 3. 比较用 `!==` 而不是 `!=`，否则输出是 X 时会被当成"相等"放过去。

5. 流程：十个步骤

步骤	目录	命令	通过标准
准备	`0_lib`	`./run.sh`	生成 `ndm/*.ndm`，日志里 `Workspace check succeeded`
1 RTL 仿真	`0_simulation_pre`	`make`；看波形 `make vd`	`TEST PASSED`
2 综合	`1_dc`	`./run.sh`	建立时间 slack ≥ 0
3 形式验证	`2_formality_postdc`	`./run.sh`	`Verification SUCCEEDED`
4 综合后仿真	`3_simulation_postdc`	`make`	`TEST PASSED`，结果与第 1 步一致
5 布局布线	`4_icc2`	`./run.sh N` 或 `./run.sh all`	时序无违例，DRC、未连通均为 0
6 形式验证	`6_formality_postlayout`	`./run.sh`	`Verification SUCCEEDED`
7 版图验证	`5_DRC`、`5_LVS`	`./run.sh`	DRC 无违例（或逐条说明原因），LVS `CORRECT`
8 寄生参数	`7_StarRC`	`./run.sh`	三个角各一份 SPEF
9 时序签核	`8_pt`	`./run.sh`	三个角建立、保持均无违例
10 后仿	`9_simulation_postlayout`	`make`	`TEST PASSED`

5.1 第 1 步：RTL 仿真

把设计放进 `0_simulation_pre/rtl/`，文件名写进 `filelist.f`，然后 `make`。脚本用 VCS 编译并运行，结尾打印检查了多少组、失败多少组、最大误差是多少个最低位。

这一步要达到的不只是"通过"：边界值（各输入的 0、最大值、余弦过零点和极值点）要专门覆盖，最大误差离 ±4 还有多少余量要心里有数，因为后面不会再有机会修功能。

5.2 第 2 步：逻辑综合

在 `1_dc/tcl/top.sdc` 里写好约束后运行 `./run.sh`。脚本读 RTL、加约束、综合，写出网表、SDF、SDC、给 Formality 用的 SVF，以及时序、面积、功耗报告。跑完停在 `dc_shell` 提示符，`exit` 退出。

看 `output/function_gen.qor.rpt`：建立时间 slack 是否为负，有多少条违例路径；再看 `timing.rpt` 里最差的几条路径落在哪个模块。综合阶段没有余量的设计，到后端只会更难。

5.3 第 3 步：综合后形式验证

Formality 以 RTL 为参考、综合网表为实现，逐个比较点证明两者等价。比较点的数量大致等于触发器数加输出端口数。通过的标志是 `Verification SUCCEEDED`，失败时看 `reports/` 下的失败点报告。

5.4 第 4 步：综合后仿真

用综合网表加标准单元模型，反标综合写出的 SDF，再跑一遍同一个 testbench。结果应该和 RTL 仿真一字不差。

5.5 第 5 步：布局布线

`4_icc2/scripts/` 下九个脚本对应九个阶段：

脚本	阶段	做完看什么
`00_init`	建设计库，读网表，读 UPF，建两个分析场景（最慢角查建立，最快角查保持）	单元数与综合报告一致
`01_floorplan`	按利用率定核心区和单元行	核心区尺寸，行高是否等于单元高度
`02_power`	电源环、电源条、单元行电源轨	电源连通性和电源网络 DRC
`03_pin`	端口位置	引脚数等于信号端口数
`04_place`	布局和布局阶段优化	建立时间；所有单元都接上电源
`05_cts`	时钟树和时钟树后的优化	时钟偏差、级数；保持时间违例数
`06_route`	布线和布线后优化	建立、保持；DRC、未连通、天线
`07_finish`	填充单元，最终检查	检查报告全部干净
`08_export`	导出网表、DEF、SPEF、GDS	`postlayout/` 下文件齐全

三种运行方式：

```
./run.sh        图形界面，在 icc2_shell 里逐个 source 脚本（需要图形桌面）
./run.sh N      批处理，从头跑到第 N 步后停在命令行
./run.sh all    批处理，从头跑到导出后退出
```

第一次做建议用 `./run.sh N` 一步步推进，每步看完报告再往下。

5.6 第 6 步：布局布线后的形式验证

以综合网表为参考、后端网表为实现。后端插了时钟树、延迟单元，还做过多轮优化，这一步证明逻辑功能没有被改坏。

5.7 第 7 步：版图验证

- **DRC**：检查版图是否满足工艺的几何规则。看汇总文件里每条规则的违例数。
- **天线检查**：通常是单独一份规则文件，`5_DRC/run.sh` 会一并运行。
- **LVS**：从版图提取出晶体管级电路，与网表比对。脚本先用 `v2lvs` 把带电源的门级网表转成 SPICE，再运行 Calibre。结论有三种：`CORRECT`、`INCORRECT`、`NOT COMPARED`（比对之前就出错了）。

5.8 第 8、9 步：寄生参数和时序签核

有 StarRC 工艺文件时用 StarRC 从版图提取寄生参数；没有时，脚本改用 ICC2 布线后提取并导出的 SPEF。PrimeTime 读后端网表和 SPEF，在三个工艺角分别检查建立和保持时间，并写出后仿用的 SDF。

这一步是时序上的最终结论。除了看有没有违例，还要看覆盖率报告，确认所有触发器和输出端口都被检查到了，"没有违例"不是因为漏查。

5.9 第 10 步：后仿

后端网表加 PrimeTime 写出的 SDF，同一个 testbench。结果与前两次一致，全流程才算闭环。

6. 题目版留给你的问题

题目版的脚本能把流程跑起来，但下面这些事没有替你做。它们都是在真实工具上做这道题时会依次撞上的问题，这里只给现象和方向。

6.1 前端

问题	现象	方向
运算怎么拆成流水线	一拍里放一个完整的乘法或除法，时序差好几纳秒	5 ns 周期扣掉 1 ns 不确定度只剩 4 ns。估一下每种运算要几拍，哪些运算可以并行，哪些互相等
余弦怎么算	4096 项的表很大	利用对称性只存一部分；想清楚输出要几位、符号怎么处理
除法怎么做	除法是整条通路上最长的一段	逐位除法每拍出一位；商的位宽取决于被除数和除数的关系，不一定要算满
精度够不够	最大误差超过 4 个最低位	每一级截断丢多少、要不要保护位、在哪里舍入，列一张误差预算表
约束怎么写	`top.sdc` 是空的	时钟、不确定度、输入输出延迟。输入输出延迟的最大值和最小值分别约束什么，要想清楚
综合后 slack 是小负数	一百多条路径都只差几十皮秒	综合工具默认只盯着最差的一条路径

6.2 后端

问题	现象	方向
库里找不到填充单元	插填充单元时报没有匹配的单元	只有物理信息、没有时序信息的单元，建库时会被怎样处理
DRC 报几万处闩锁类违例	几乎每个单元都报	看规则原文在检查什么；看库里有没有为此准备的单元，它该在哪个阶段放进去
天线违例	布局布线工具自查没问题，Calibre 报违例	布局布线工具知不知道天线规则；规则文件里的比例怎么换算成工具的设置
输入端口的线仍有天线违例	加了天线规则后，剩下几条都在芯片边缘	这些线网和内部线网的差别是什么；库里有专门的单元
布线后剩几十皮秒违例	建立和保持各有几条，再优化一轮也不动	让工具在优化时比签核标准多做一点
金属密度违例	每层金属各报一条	这是模块级设计都会有的，要能说清原因和实际项目里的解决办法
LVS 结论是 NOT COMPARED	网表读入阶段就报错	看报告开头说找不到什么；那些单元里有没有器件
LVS 结论是 INCORRECT	器件数差几十个，错误集中在个别模块	留意出错线网的名字有什么共同点；Verilog 和 SPICE 在名字上的规则有什么不同

问题	现象	方向
PrimeTime 与布局布线工具结论不一致	后者没有违例，前者在端口路径上差半纳秒	时钟树做完后，两个工具各自怎样看待端口处的时钟；回头检查输入输出延迟的写法
最慢角用错了寄生参数	三个角的 SPEF 一模一样	不同温度下连线电阻不同，核对每个角读的是哪份文件

6.3 做题的顺序建议

1. 先用高级语言或 testbench 里的实数模型把算法和误差预算定下来，再写 RTL。
2. 每写完一个模块就单独仿真，不要等顶层连起来再查。
3. 第一次综合就看时序，流水线划分有问题越早发现越好。
4. 后端一步一步推进，每个阶段都看报告；出了问题先弄清楚是设计的问题、约束的问题还是脚本的问题。
5. 约束或 RTL 改了以后，从综合开始整条流程重跑，不要只重跑出问题的那一步。

7. 参考结果

`solution` 分支里的参考设计在上述工具上从头到尾跑通，结果如下，可以作为对照。

项目	结果
结构	18 拍流水线：输入寄存 1 拍，乘法 / 余弦查表 / 除数准备并行 4 拍，逐位除法并同时累加 12 拍，输出 1 拍
功能	102 376 组向量（边界值加随机）全部通过，最大误差 1.891 个最低位
形式验证	综合后、布局布线后各 1264 个比较点，全部等价
综合	约 6400 个单元，其中触发器 1251 个
版图	约 584 × 581 μm，核心区利用率 40%，标准单元面积约 127 000 μm^2
时序签核	三个工艺角无违例；最慢角建立时间余量 +0.020 ns，最快角保持时间余量 +0.047 ns
DRC	只剩 6 条金属最小密度规则（模块级没有做金属填充）
天线、LVS	0 违例；`CORRECT`
三次仿真	RTL、综合后、布局布线后结果完全一致

读这组数据时要知道的

- 建立时间余量只有 20 皮秒，是扣除 1 ns 不确定度之后的值。设计满足 200 MHz，但没有富余。
- 输入输出延迟各取半个周期，没有按"输入紧贴时钟沿变化"的最严格情况约束。
- 寄生参数来自布局布线工具的提取，不是签核级的提取工具；连线寄生只有典型工艺一种模型，三个角的差别体现在单元延迟和温度上。
- 18 拍不是下限。乘法和除法的拍数、每拍的逻辑深度之间还有调整空间。

8. 说明

- 题目来自一门数字集成电路设计课程的课程设计，这里按自己的理解重新表述；课程方的讲义和手册不在仓库里。
- 仓库不含任何工艺厂或 EDA 厂商的文件，也不含由它们生成的结果文件；脚本里出现的库名和单元名只是配置示例，需要换成你自己的。
- 脚本和参考设计是在一套具体的工具版本和工艺库上调通的，换环境后预计需要调整，尤其是布局布线工具的选项名、库的单元名，以及版图验证规则文件的用法。